Enterprise Application Development 2024B

Lab Test 2

Total marks: 100

Reading Time**: 15 minutes**
Working Time**: 120 minutes**
Submission Time**: 15 minutes**

Rules:

1. Students are allowed use online Search tools (e.g., Google, Bing) for reference
2. Students MUST NOT use AI assistant tools such as Copilot and ChatGPT.
3. No discussion or exchange of ideas. External support is strongly prohibited.
4. Students put all the answer files into a folder, zip it, and submit the zipped file.
5. Sharing or disclosing lab test questions to forums or Q&A platforms is prohibited.
6. Students should make their assumptions based on common sense. Please don't ask questions that are straightforward as it can slow down your and others' progress.
7. **Students can use any HTML, CSS, or Javascript framework to do this question**
8. **Students can reuse code from tutorials, mini-tests, project or self-practice programs, provided that these codes do not directly solve the lab test 2 problem.**

Test Description

Mr. Tu is planning to open a start-up in the real-estate market. He wants to have a full-stack web application for managing his real-estate properties. He seeks your help to build a prototype for testing his business concepts.

The test has four levels of requirements.

Visual Appealing is a factor **used in evaluating excellent work**. **In this test, you should prioritize functionality over appearance**.

Level 0 Requirements (7 points)

- Your solution shall contain the **frontend** and **backend** folders, where the **frontend folder is NOT inside the backend folder**. The basic architecture requirements are as follows:

I. Frontend

- API calls to the backend must be separated from the UI

- **Inline styling is NOT permitted**. E.g., **<h2 ~~style=”color:red”~~>**

- **UI is neatly and intuitively designed**

II. Backend

1. You must use the **Spring Boot pattern (Controller, Service, Repository, and Model)** in the backend.

2. You must use H2, Postgres, MySQL, or MongoDB for storing the data. The database can be setup locally or on the cloud. **Your backend must setup the database automatically** without running any queries to create a database/schema and table/collection.

3. If using Postgres, store your data under the postgres database and public schema. Not meeting this requirement results in a 3-point penalty for the entire test.

4. If using H2, MySQL, or MongoDB, store your data under a labtest2 database. Not meeting this requirement results in a 3-point penalty for the entire test.

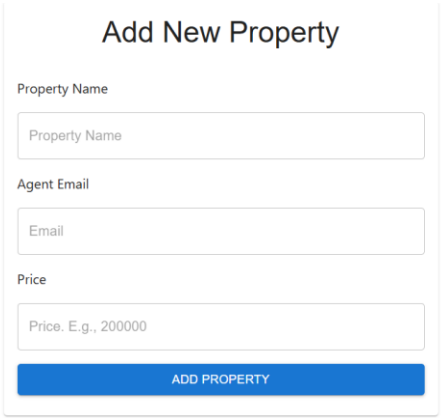
Precautions for Level 2 and 3

1. Your backend shall apply the **Modular Monolith architecture**
2. You must use **ResponseEntity** for every response from your Controller
3. You must apply **Optional** for every search, data retrieval (GET) and update (PUT/PATCH) to handle null cases in your backend.
4. Your frontend must define a dedicated utility function for **handling HTTP REST Request**s, i.e., GET, POST, PUT, or DELETE requests.
5. Implement a **Service Interface** using the Java Interface feature, and a **GET Endpoint** to get the ID, name, agent email, and price of a Property by its ID

Level 1 Requirements (48 points)

I. Frontend (30 points)

You will develop **a Catalogue Page** for listing and creating real-estate properties as shown in Figure 1.



ID	Name	Agent Email	Price
1	John Smith House	john.smith@example.com	$529,030,398
2	Jane Johnson House	jane.johnson@example.com	$530,212,955
3	Jane Williams House	jane.williams@example.com	$172,379,904
4	Emily Williams House	emily.williams@example.com	$158,917,352
5	William Brown House	william.brown@example.com	$428,112,852
6	Olivia Davis House	olivia.davis@example.com	$472,731,785
7	James Miller House	james.miller@example.com	$19,432,148
8	Sophia Wilson House	sophia.wilson@example.com	$684,083,656
9	Benjamin Moore House	benjamin.moore@example.com	$1,055,949,840
10	Isabella Taylor House	isabella.taylor@example.com	$764,271,163

Figure 1: A Basic Property Page that offer real-estate property creation and listing

A. Show Properties

Show all the real-estate properties as a table.

1. Content

- Each property is represented as a row containing the following data:
    a) **ID**
    b) **Name**
    c) Real-estate Agent **Email** address. Do NOT create an Entity for Real Estate Agents
    d) **Price as an integer** in dollars. For example: 1,000,000; 325,000

To foster testability, you are required to name the 3 variables in your backend entity **EXACTLY** as **name**, **agentEmail**, and **price**. **Failure to address this requirement results in up to 3 points penalty.**

2. User Interface

- The table is **center-aligned**
- The table has four columns with gap in the middle
- A Panel on top of the page for creating a new Property
- The table width **is 70-80%** of the page width
- The price is comma-separated for values larger than 999, prepended by the $ sign.

B. Add New Property

1. The **Property Creation Panel** provides options for inputting **Name**, **Agent Email**, and **Price**
2. Each field should be presented in a separate line, as shown in Figure 1.
3. The form has a **Create** button to send a POST request for creating the new property
4. After the property is created in the backend, pop-up an alert of the successful creation as shown in Figure 2.
5. After a successful creation, the frontend must **request the backend** to get the latest list of properties, including the new one, and show them on the table **without refreshing the page**.

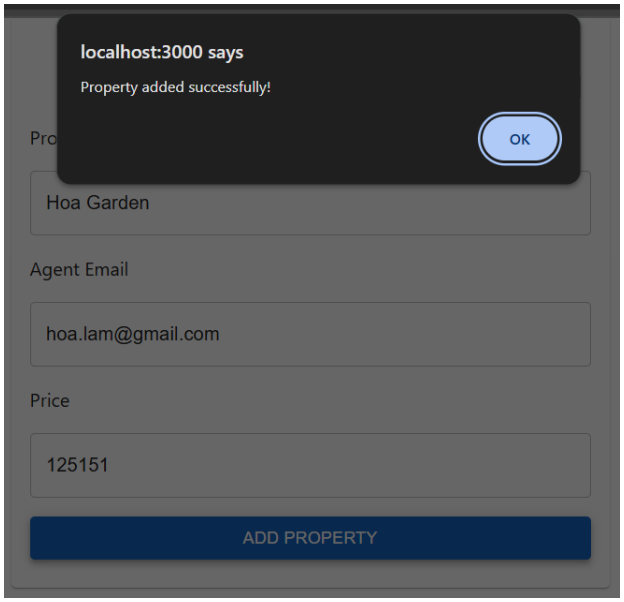


Figure 2: Show Success Message and append a new row after creating the new Property

II. Backend (18 points)

Your solution must apply the Spring Boot pattern of using the Controller, Service, Repository, and Model classes.

1. Provide a GET endpoint to show all real-estate properties as described in I.A.1. **The data of the properties must be retrieved from the database**.

2. Provide a POST endpoint to add a new property to the database. **Do NOT create a new property if there exists another property with the same name as the new one.**

Level 2 Requirements (15 points)

I. Frontend (10 points)

Provide the **Updating** and **Deleting** Properties as shown in Figure 3.

ID	Name	Agent Email	Price		
1	John Smith House	john.smith@example.com	$35,949,888	Delete	Edit
2	Jane Johnson House	jane.johnson@example.com	$46,692,474	Delete	Edit
3	Jane Williams House	jane.williams@example.com	$1,164,336,186	Delete	Edit
4	Emily Williams House	emily.williams@example.com	$137,842,714	Delete	Edit

Figure 3: Insert two buttons for Deleting and Updating a property

A. REST Request Utility Function

Your **frontend** must define a dedicated utility function for **handling HTTP REST Requests**, i.e., sending GET, POST, PUT, or DELETE requests to the backend using this utility function.

B. Delete a Property

- Add a Delete Button in every row of the product
- Upon clicking the Delete button, the frontend sends a DELETE REST request to remove the property from the database at the backend
- When the backend confirms that the property is deleted, **the frontend requests the new list of properties from the backend** and reload those properties **without refreshing the page**.

C. Update a Property

- Add an Update Button in every row of the product
- Upon clicking the Update Button, an Update UI loads the data of the chosen property into the corresponding input fields

- The Heading is **"Update Property [propertyID]"**
- The Confirm button text is **"Update"**
- **Display the success message "Successfully update property",** as shown in Figure 4
- Right after updating the property, **the frontend must request the most recent list of properties from the backend** and populate those properties on the table **without refreshing the page**

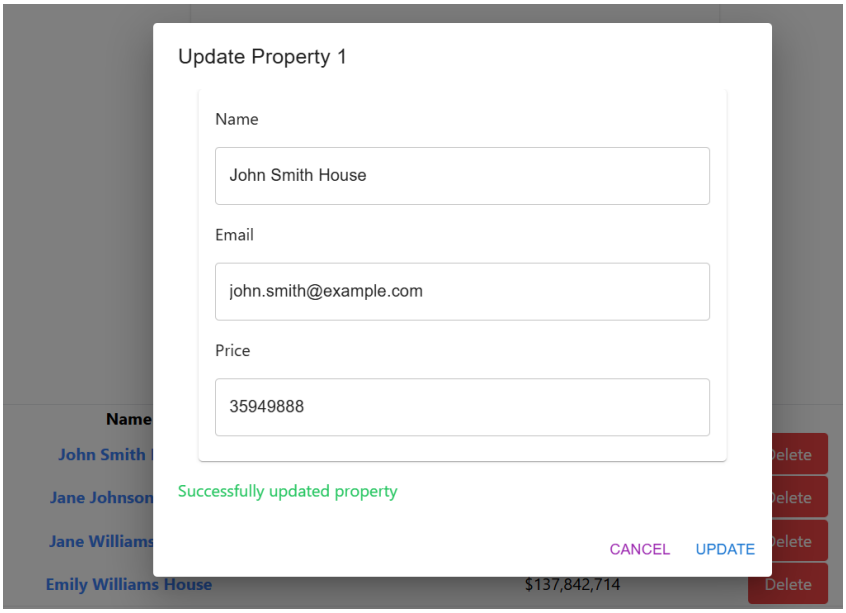


Figure 4: Modal Update pre-loaded with property data. The changes are immediately updated in the table without reloading the page

II. Backend (5 points)

1. Provide a DELETE endpoint to delete a product by ID. **You must check that the deleted real-estate property does not exist in the database after the delete statement is executed.**

2. Provide a PUT endpoint to update a property based on the property object sent from the frontend. **You must update only when the to-be-updated property exists in the database**.

3. Implement a **Service Interface** using the Java Interface feature, and a **GET Endpoint** to get the ID, name, agent email, and price of a Property by its ID

Level 3 Requirements (30 points)

I. Architecture Requirements (5 points)

1. Your backend shall apply the **Modular Monolith architecture.**

- Only the **Service Interface in Level 2.II.3** and **Property Model** are *publicly accessible*
- **Other Services, Controller** and **Repository classes** along with **their functions and variables** are *only visible within the module*

2. You must use **ResponseEntity** for every response from your Controller

3. You must apply **Optional** for every data retrieval (GET) and update (PUT/PATCH) to handle null cases in your backend. **The Controller shall check the Optional object to return an OK or error response.**

II. Front- and Back-end Validations (8 points)

Validate the **Agent Email** and **Price** on both the frontend and backend when **Creating** and **Updating** Properties (See Figure 5 and 6). The rules are as follows:

1. **Agent Email** must follow the pattern **prefix@domain**
   - The **prefix** only contains **alphanumeric characters** and **period** (.), **dash** (-), or **underscore** (_)
   - The **domain** must **end with .com**

2. **Price** must range from 0 to 2 billion

A. Frontend Validation on Creation and Update (6 points)

- The UI must display the error message **near the erroneous field**, indicating the **valid rule** and an **example** of a **valid value**. For example:

  The price must range from 0 to 2 billion, e.g., 200,000

- The validation can be activated **in real-time at every input event** OR **upon submission**.

B. Backend Validation (2 points)

- Any request for **creating or updating a property** with invalid price or agent email **must be rejected with a Bad Request Status**.

- The response body shall indicate the name of each invalid attribute and its error cause.

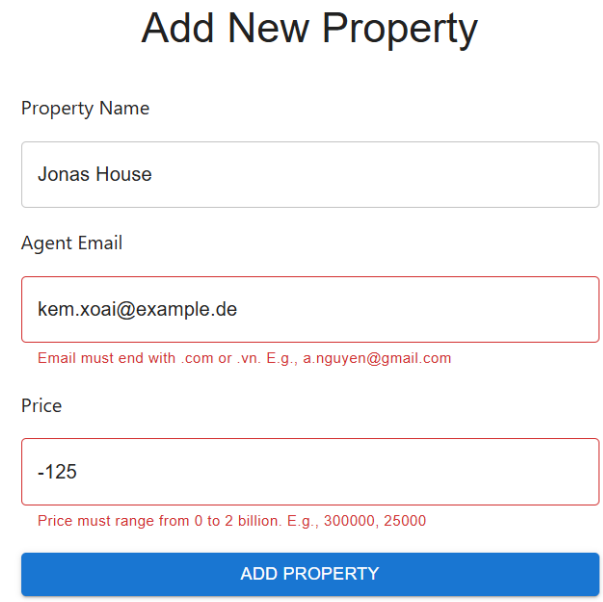


Figure 5: Error message shown beneath the fields in Creating Properties with explanation of the error cause and an example of a valid value

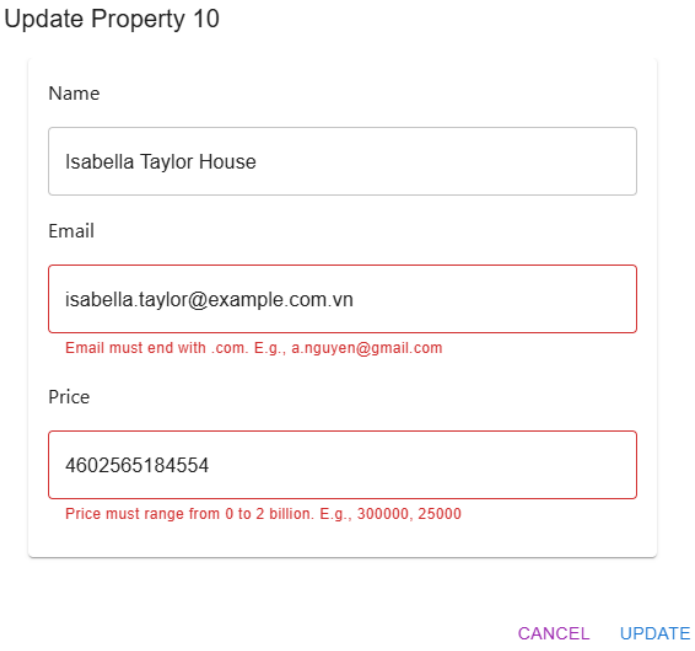


Figure 6: Error message shown beneath the fields in Updating Properties with explanation of the error cause and an example of a valid value

III. Pagination and Sorting (11 points)

Display properties by **pages** with **sort options** by *name*, *ID*, or *price*.

A. Pagination (8 points)

- On page load, paginate the result with **4 properties per page.**
- Your web page must indicate which page the user is currently viewing
- The frontend must show all the available pages below the table.
- The user can click on the **page option** or the **previous/next button** to navigate across pages
- Figure 7 shows the sample UI design for offering pagination.
- **The Pagination of data must be done in the backend using Spring JPA features.**

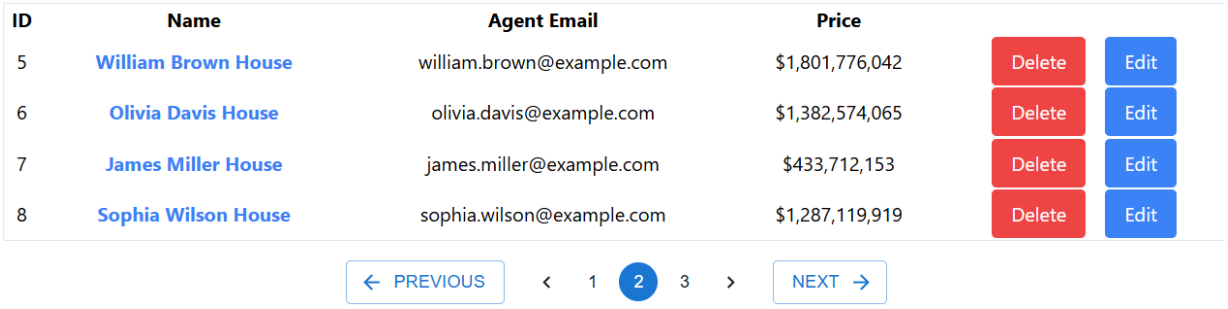


Figure 7: Pagination of the products, with the active page and all possible pages

B. Sorting (3 points)

- Provides a dropdown button for the user to sort the property list by the **ID (default or None option)**, **name**, **agent email**, OR **price** in **DESCENDING** order (See **Figure 8**).

- **The sorting must be done in the backend using Spring JPA features.**

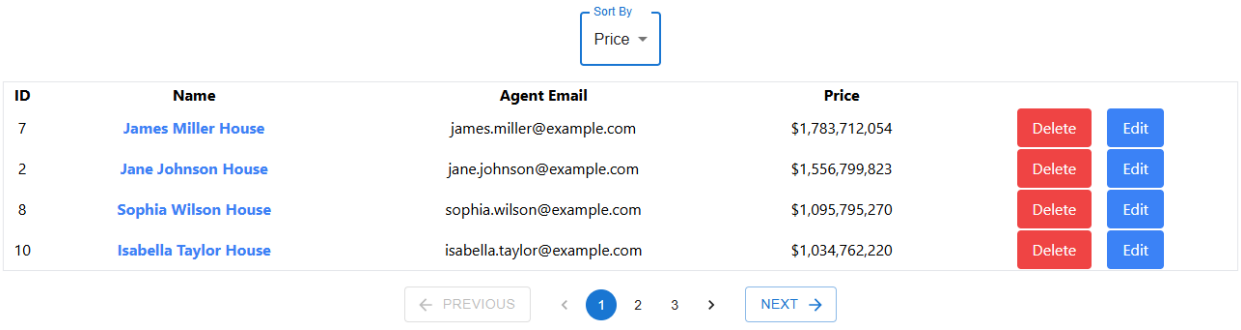


Figure 8: Offer a Sort by Dropdown button for sorting the table in descending order by ID, name, email, or price. If no option is selected, the table is sorted by ID in descending order.

IV. Authorization (6 points)

Secure your application such that only an Admin can access the above features of managing properties.

A. Admin Credential and Login (3 points)

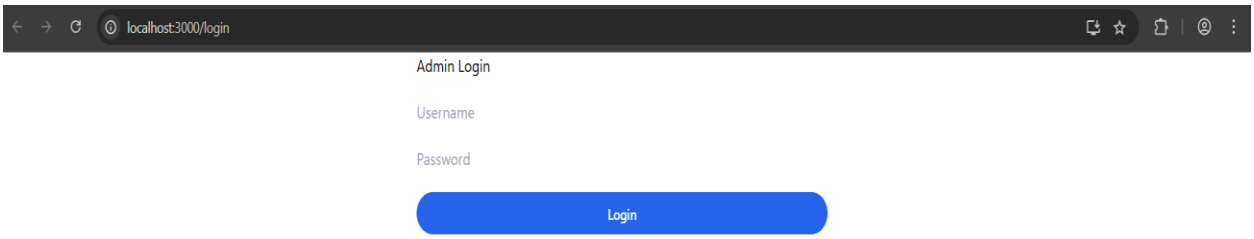
- Provide an Admin registration endpoint that takes an **email**, **password, and role** in the request body.

- Provide a Login endpoint that takes an **email** and **password** to authenticate the Admin.

You are NOT required to develop an interface for registering an Admin.

B. Authorize Property Management Page (3 points)

- Provide a Login Page like below when loading your frontend application.



- **Only a valid and authenticated admin can access the property management page** for CRUD properties.

END OF LAB TEST 2 DESCRIPTION